TCC805x MCU BSP

API Specification

for Power Management Input Output

Rev. 1.30 [A]

2021-06-01

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit http://www.telechips.com for more information.

TABLE OF CONTENTS

Contents

Figures

Tables

1 INTRODUCTION

This document describes the power management input output device driver (hereinafter referred to as PMIO driver) of TCC805x MCU BSP. For more detailed information of the power management input output in the TCC805x MCU Sub-system, refer to Chapter 23.3 Power Management I/O (PMIO) of the *"PART 11-MICOM SUB-SYSTEM"* in *"TCC805x Full Specification"*. [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

PMIO	Power Management Input Output
STR	Suspend To RAM
BU	Battery Unit or B+
RTC	Real Time Clock
AP	Application Processor (CA72, CA53 core)
MBOX	Mailbox

3 SOURCE FILES

3.1 Location of Source Files

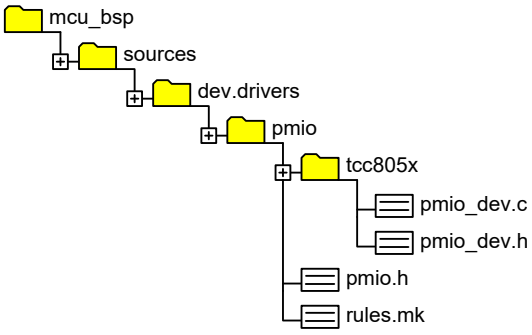

Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- pmio_dev.c
  - Power manager driver source
- pmio_dev.h
  - Power manager driver header
- pmio.h
  - Power manager header for functional configuration, parameter define, and interface
  - Power function configurations (STR, CAN detection, and so on). Refer to Chapter 7.1 Set Configuration.
- rules.mk
  - Rules used for build

4 ENUMERATIONS

4.1 PMIOMode_t

Enumeration variables to manage power mode. This value is used to perform power-down or to inform the AP of the power mode in STR mode.

Declaration

```
typedef enum
{
    PMIO_MODE_NOT_SET     = 0,
    PMIO_MODE_POWER_ON    = 1,
    PMIO_MODE_ACTIVE      = 2,
    PMIO_MODE_STR         = 3,
    PMIO_MODE_POWER_DOWN  = 4,
    PMIO_MODE_MAX         = 5
} PMIOMode_t;
```

Description

Value	Description
PMIO_MODE_NOT_SET	Power mode is not set
PMIO_MODE_POWER_ON	Power-on status value
PMIO_MODE_ACTIVE	Active status value
PMIO_MODE_STR	Suspend to RAM status value
PMIO_MODE_POWER_DOWN	Power-down status value
PMIO_MODE_MAX	Maximum value of power mode

Remark

None

4.2 PMIOReason_t

Enumeration variables to manage power-on reason state. This value is used to inform the AP of the power-on reason in the STR mode.

Declaration

```
typedef enum
{
    PMIO_REASON_NOT_SET   = 0,
    PMIO_REASON_BU_ON     = 1,
    PMIO_REASON_ACC_ON    = 2,
    PMIO_REASON_CAN_ON    = 3,
    PMIO_REASON_RTC_ON    = 4,
    PMIO_REASON_MAX       = 5
} PMIOReason_t;
```

Description

Value	Description
PMIO_REASON_NOT_SET	Power-on reason is not set.
PMIO_REASON_BU_ON	Power-on reason is BU.
PMIO_REASON_ACC_ON	Power-on reason is ACC ON.
PMIO_REASON_CAN_ON	Power-on reason is CAN ON.
PMIO_REASON_RTC_ON	Power-on reason is RTC wake-up.
PMIO_REASON_MAX	Maximum value of Power-on reason

Remark

None

4.3 PMIOMsg_t

Enumeration variables to manage message of power state for other APs. This value is only used when communicating with AP in STR mode.

Declaration

```
typedef enum
{
    PMIO_MSG_NOT_SET          = 0,
    PMIO_MSG_ACK              = 1,    //used (ALL)(AP->R5 : for first MBOX ready check)
    PMIO_MSG_ACC_ON           = 2,    //used      (R5->AP)
    PMIO_MSG_STR              = 3,    //used      (R5->AP)
    PMIO_MSG_STR_DONE         = 4 ,   //used      (AP->R5)
    PMIO_MSG_POWER_DOWN       = 5,    //not used  (R5->AP)
    PMIO_MSG_POWER_DOWN_DONE  = 6,    //not used  (AP->R5)
    PMIO_MSG_MAX              = 7
} PMIOMsg_t;
```

Description

Value	Description
PMIO_MSG_NOT_SET	PMIO message for STR mode
PMIO_MSG_ACK	PMIO message for STR mode This message is used to check whether the MBOX between MCU and main core is ready.
PMIO_MSG_ACC_ON	PMIO message for STR mode The power manager delivers this message to the main core when the ACC ON signal is input to the MCU.
PMIO_MSG_STR	PMIO message for STR mode The power manager delivers this message to the main core when changing to STR mode.
PMIO_MSG_STR_DONE	PMIO message for STR mode The power manager receives this message from the main core.
PMIO_MSG_POWER_DOWN	PMIO message is not yet used.
PMIO_MSG_POWER_DOWN_DONE	PMIO message is not yet used.
PMIO_MSG_MAX	Maximum value of PMIO message type

Remark

None

4.4 PMIOPortSel_t

Enumeration variables to manage GPIO_K port. This value is only used to control the GPIO_K/PCTL port.

Declaration

```
typedef enum
{
        PMIO_PORT_SEL_RELEASE           = 0,

        PMIO_PORT_SEL_GPIO              = 1,

        PMIO_PORT_SEL_PMGPIO_IN         = 2,
        PMIO_PORT_SEL_PMGPIO_OUT_L      = 3,
        PMIO_PORT_SEL_PMGPIO_OUT_H      = 4,
        PMIO_PORT_SEL_PMGPIO_PU         = 5,
        PMIO_PORT_SEL_PMGPIO_PD         = 6,

        PMIO_PORT_SEL_PCTLIO_IN         = 7,
        PMIO_PORT_SEL_PCTLIO_OUT_L      = 8,
        PMIO_PORT_SEL_PCTLIO_OUT_H      = 9,
        PMIO_PORT_SEL_PCTLIO_PU         = 10,
        PMIO_PORT_SEL_PCTLIO_PD         = 11
} PMIOPortSel_t;
```

Description

Value	Description
PMIO_PORT_SEL_RELEASE	The GPIO_K port with this value selected is controlled by PMIO and is set to the input disable state.
PMIO_PORT_SEL_GPIO	The GPIO_K port with this value selected is controlled by GPIO. Use the GPIO driver to control this port.
PMIO_PORT_SEL_PMGPIO_IN	The GPIO_K port with this value selected is controlled by PMIO and is set to the input enable state.
PMIO_PORT_SEL_PMGPIO_OUT_x	The GPIO_K port with this value selected is controlled by PMIO and is set to the output low/high state. ■ PMIO_PORT_SEL_PMGPIO_OUT_L: low ■ PMIO_PORT_SEL_PMGPIO_OUT_H: high
PMIO_PORT_SEL_PMGPIO_Px	The GPIO_K port with this value selected is controlled by PMIO. ■ PMIO_PORT_SEL_PMGPIO_PU: Pull-up ■ PMIO_PORT_SEL_PMGPIO_PD: Pull-down
PMIO_PORT_SEL_PCTLIO_IN	The PCTL port with this value selected is controlled by PMIO and is set to the input enable state.
PMIO_PORT_SEL_PCTLIO_OUT_x	The PCTL port with this value selected is controlled by PMIO and is set to the output low/high state. ■ PMIO_PORT_SEL_PCTLIO_OUT_L: low ■ PMIO_PORT_SEL_PCTLIO_OUT_H: high
PMIO_PORT_SEL_PCTLIO_Px	The PCTL port with this value selected is controlled by PMIO.

Value	Description
	■ PMIO_PORT_SEL_PCTLIO_PU: Pull-up ■ PMIO_PORT_SEL_PCTLIO_PD: Pull-down

Remark

None

4.5 PMIOTimerReason_t

Enumeration variables to manage the reason for timer operation. This value is only used to control timer in PMIO.

Declaration

```
typedef enum
{
    PMIO_TIMER_REASON_NOT_SET           = 0,
    PMIO_TIMER_REASON_ACC_OFF           = 1,
    PMIO_TIMER_REASON_STR_REQ           = 2
} PMIOTimerReason_t;
```

Description

Value	Description
PMIO_TIMER_REASON_NOT_SET	Reason for timer operation is cleared. This value is used when the timer operation is disabled.
PMIO_TIMER_REASON_ACC_OFF	When the system is powered on by BU ON and in ACC OFF state, the timer will have this value.
PMIO_TIMER_REASON_STR_REQ	When MCU does not receive the AP's STR_DONE message, the timer has this value as a reason.

Remark

None

5 CONSTANTS

5.1 PMIO Interrupt Source Number

Definition variables of GIC's PMIO interrupt number

Declaration

```
#define PMIO_VA_INTERRUPT_SRC          ((uint32)(GIC_PMGPIO))
#define PMIO_VA_RTC_INTERRUPT_SRC      ((uint32)(GIC_RTC))
```

Description

Value	Description
PMIO_VA_INTERRUPT_SRC	PMIO interrupt source number
PMIO_VA_RTC_INTERRUPT_SRC	RTC interrupt source number (required in STR mode)

Remark

Refer to "*TCC805x Full Specification*" about control registers offset in each base address.

5.2 PMIO Utility Function

Definition variables of PMIO's Macro Function

Declaration

```
#define PMIO_GPK(x)                   ((uint32)1<<(uint32)x)
#define PMIO_FUNC_MSEC_TO_USEC(x)      ((uint32)(x) * (uint32)(1000UL))
#define PMIO_FUNC_SEC_TO_USEC(x)       ((uint32)(x) * (uint32)(1000UL) * (uint32)(1000UL))
```

Description

Value	Description
PMIO_GPK(x)	GPIO_K bit flag
PMIO_FUNC_xSEC_TO_USEC(x)	Time value calculator

Remark

Refer to "*TCC805x Full Specification*" about control registers offset in each base address.

5.3 PMIO Port

Definition variables to manage bits for PMGPIO port

Declaration

```
#define PMIO_VA_PCTL_0_PORT          (PMIO_VA_BIT0)
#define PMIO_VA_PCTL_1_PORT          (PMIO_VA_BIT1)

#define PMIO_VA_BU_DET_PORT          (PMIO_VA_BIT1)
#define PMIO_VA_ACC_DET_PORT         (PMIO_VA_BIT3)
#define PMIO_VA_CAN_DET_PORT         (PMIO_VA_BIT14)
#define PMIO_VA_STR_DET_PORT         (PMIO_VA_BIT12)
```

Description

Value	Description
PMIO_VA_[Name]_PORT	In Telechips Evaluation Board (EVB) environment, ■ **BU det** is set to **GPK01**. ■ **ACC_DET** is set to **GPK03**. ■ **PCTL0** is set to **PCTL0**. ■ **PCTL1** is set to **PCTL1**.

Remark

None

5.4 PMIO Register

Definition variables to manage PMIO register address

Declaration

```
#define PMIO_VA_BASE_ADDR    TCC_MICOM_PMIO_BASE   //R/W, Backup RAM Register
#define PMIO_VA_BACKUP_RAM_ADDR  0x1B937000UL //R/W, Backup RAM Register
#define PMIO_VA_PMGPIO_CTL_ADDR  0x1B937828UL //PMGPIO Control Register
#define PMIO_VA_RST_STS0_ADDR    0xC003F400UL //Reset Status 0 Register (0x14400130)
#define PMIO_VA_RST_STS1_ADDR    0xC003F404UL //Reset Status 1 Register (0x14400134)
#define PMIO_VA_RST_STS2_ADDR    0xC003F408UL //Reset Status 2 Register (0x14400138)

/*PMIO REGISTERS*/
/*PMIO reg: GPK by PMIO*/
#define PMIO_REG_GPK_FDAT        *(SALReg32 *)0x1B937A00UL //GPK Filtered Data Port
#define PMIO_REG_GPK_IRQST       *(SALReg32 *)0x1B937A10UL //GPK Interrupt Status Register
#define PMIO_REG_GPK_IRQEN       *(SALReg32 *)0x1B937A14UL //GPK Interrupt Enable Register
#define PMIO_REG_GPK_IRQPOL      *(SALReg32 *)0x1B937A18UL //GPK Interrupt Polarity Register
#define PMIO_REG_GPK_IRQTM0      *(SALReg32 *)0x1B937A1CUL //GPK Interrupt Trigger Mode 0 Reg
#define PMIO_REG_GPK_IRQTM1      *(SALReg32 *)0x1B937A20UL //GPK Interrupt Trigger Mode 1 Reg
#define PMIO_REG_GPK_FCK         *(SALReg32 *)0x1B937A24UL //GPK Filter Clock Conf Reg
#define PMIO_REG_GPK_FBP         *(SALReg32 *)0x1B937A28UL //GPK Filter Bypass Register
#define PMIO_REG_GPK_CTL         *(SALReg32 *)0x1B937A2CUL //GPK Miscellaneous Control Reg

/*PMIO reg: Backup Ram*/
#define PMIO_REG_BACKUP_RAM      *(SALReg32 *)0x1B937000UL //R/W, Backup RAM Register

/*PMIO reg: PMGPIO*/
#define PMIO_REG_PMGPIO_DAT      *(SALReg32 *)0x1B937800UL //PMGPIO Data Reg
#define PMIO_REG_PMGPIO_EN       *(SALReg32 *)0x1B937804UL //PMGPIO Direction Control Reg
#define PMIO_REG_PMGPIO_FS       *(SALReg32 *)0x1B937808UL //PMGPIO Function Select Reg
#define PMIO_REG_PMGPIO_IEN      *(SALReg32 *)0x1B93780CUL //PMGPIO Input Enable Reg
#define PMIO_REG_PMGPIO_PE       *(SALReg32 *)0x1B937810UL //PMGPIO PULL Down/Up Enable Reg
#define PMIO_REG_PMGPIO_PS       *(SALReg32 *)0x1B937814UL //PMGPIO PULL Down/Up Select Reg
#define PMIO_REG_PMGPIO_CD0      *(SALReg32 *)0x1B937818UL //PMGPIO Driver Strength 1 Reg
#define PMIO_REG_PMGPIO_CD1      *(SALReg32 *)0x1B93781CUL //PMGPIO Driver Strength 1 Reg
#define PMIO_REG_PMGPIO_EE0      *(SALReg32 *)0x1B937820UL //PMGPIO Event Enable 0 Reg
#define PMIO_REG_PMGPIO_EE1      *(SALReg32 *)0x1B937824UL //PMGPIO Event Enable 1 Reg
#define PMIO_REG_PMGPIO_CTL      *(SALReg32 *)0x1B937828UL //PMGPIO Control Reg
#define PMIO_REG_PMGPIO_DI       *(SALReg32 *)0x1B93782CUL //PMGPIO Input Raw Status Reg
#define PMIO_REG_PMGPIO_STR      *(SALReg32 *)0x1B937830UL //PMGPIO Rising Event Status Reg
#define PMIO_REG_PMGPIO_STF      *(SALReg32 *)0x1B937834UL //PMGPIO Falling Event Status Reg
#define PMIO_REG_PMGPIO_POL      *(SALReg32 *)0x1B937838UL //PMGPIO Event Polarity Reg
#define PMIO_REG_PMGPIO_PROT     *(SALReg32 *)0x1B93783CUL //PMGPIO Protect Reg
#define PMIO_REG_PMGPIO_APB      *(SALReg32 *)0x1B937900UL //PMGPIO & PMRAM APB Timing Reg

/*PMIO reg: PCTLIO*/
#define PMIO_REG_PCTLIO_DAT      *(SALReg32 *)0x1B937840UL //PCTLIO Data Reg
#define PMIO_REG_PCTLIO_EN       *(SALReg32 *)0x1B937844UL //PCTLIO Direction Control Reg
#define PMIO_REG_PCTLIO_FS       *(SALReg32 *)0x1B937848UL //PCTLIO Function Select Reg
#define PMIO_REG_PCTLIO_IEN      *(SALReg32 *)0x1B93784CUL //PCTLIO Input Enable Reg
#define PMIO_REG_PCTLIO_PE       *(SALReg32 *)0x1B937850UL //PCTLIO PULL Down/Up Enable Reg
#define PMIO_REG_PCTLIO_PS       *(SALReg32 *)0x1B937854UL //PCTLIO PULL Down/Up Reg
#define PMIO_REG_PCTLIO_CD0      *(SALReg32 *)0x1B937858UL //PCTLIO Driver Strength 0 Reg
#define PMIO_REG_PCTLIO_CD1      *(SALReg32 *)0x1B93785CUL //PCTLIO Driver Strength 1 Reg
```

Description

Value	*Description*
PMIO_VA_[Name]_ADDR	*Register address*
PMIO_REG_[Name]	*Register*

Remark

None

5.5 PMIO Backup RAM

Definition variables of PMIO backup RAM size

Declaration

```
#define PMIO_VA_BACKUP_RAM_SIZE     1024 // 1 Kbyte Backup RAM
```

Description

Value	*Description*
PMIO_VA_BACKUP_RAM_SIZE	*Size of backup RAM in PMIO.*

Remark

None

6 APIs

6.1 PMIO_EarlyWakeUp

This function is called in ***startup.S***, for PMIO default settings in the initial stage of booting before BSP initialization. If early setting is required in user environment, it can be implemented in this way.

Declaration

```
void PMIO_EarlyWakeUp
(
    void
);
```

Parameters

None

Return

None

Remark

None

6.2 PMIO_Init

This function is called from ***bsp.c***, and PMIO is initialized during BSP initialization.

Declaration

```
void PMIO_Init
(
    void
);
```

Parameters

None

Return

None

Remark

None

6.3 PMIO_PowerDown

This function is called by interrupt in ***pmio_dev.c*** and performs power-down when an interrupt for ACC and BU occurs.

Declaration

```
void PMIO_PowerDown
(
    PMIOMode_t     tMode
);
```

Parameters

Value	Description
tMode	Power-down mode type Depending on the scenario, you can power down or suspend the system. In the normal power-down scenario (STR mode: disabled), PMIO_MODE_POWER_DOWN can be passed as an input parameter. In the Suspend and Power down scenario (STR mode: enabled), PMIO_MODE_POWER_DOWN or PMIO_MODE_STR can be passed as an input parameter.

Return

None

Remark

None

6.4 PMIO_GetBackupRam

This function reads the backup RAM register.

Declaration

```
uint32 PMIO_GetBackupRam
(
    uint32     uiAddr
);
```

Parameters

Value	Description
uiAddr	Backup RAM register address

Return

Value	Description
uint32	Backup RAM register value

Remark

None

6.5 PMIO_SetBackupRam

This function writes the backup RAM register.

Declaration

```
sint8 PMIO_SetBackupRam
(
    uint32      uiAddr,
    uint32      uiVa
);
```

Parameters

Value	Description
uiAddr	Backup RAM register address
uiVa	Backup RAM register value

Return

Value	Description
0	Success
1	Failure

Remark

None

6.6 PMIO_ClearBackupRAM

This function clears the data in backup RAM register.

Declaration

```
void PMIO_ClearBackupRAM
(
    void
);
```

Parameters

None

Return

None

Remark

None

6.7 PMIO_GetBootReason

This function allows you to distinguish between power-on by BU, ACC, CAN, and RTC.

Declaration

```
PMIOReason_t PMIO_GetBootReason
(
    void
);
```

Parameters

None

Return

Value	Description
PMIOReason_t	Reason type

Remark

None

6.8 PMIO_DebugPrintPowerStatus

This function is for debugging the power status.

Declaration

```
void PMIO_DebugPrintPowerStatus
(
    void
);
```

Parameters

None

Return

None

Remark

None

6.9 PMIO_SetGPK

This function allows you to distinguish between power-on by BU, ACC, CAN, and RTC.

Declaration

```
void PMIO_SetGPK
(
    uint32            uiPort32
);
```

Parameters

Value	Description
uiPort32	32-bit for GPIO_K[18:0] pin **Example:** GPIO_K_1: uiPort32 = 0x1 GPIO_K_3: uiPort32 = 0x8 or (0x1<<3) GPIO_K_1 and GPIO_K_3: uiPort32 = 0x9

Return

None

Remark

None

6.10 PMIO_GetWdtReset

This function can check whether the watchdog timer is reset or not.

Declaration

```
uint8 PMIO_GetWdtReset
(
    void
);
```

Parameters

None

Return

Value	Description
uint8	0: No Watchdog timer reset 1: Watchdog timer reset occurs

Remark

None

6.11 PMIO_ForceSTRModeTest

This function is only for internal STR testing.

Declaration

```
void PMIO_ForceSTRModeTest
(
    void
);
```

Parameters

None

Return

None

Remark

None

6.12 PMIO_APBootOk

This function is only for internal STR testing.

Declaration

```
void PMIO_APBootOk
(
    void
);
```

Parameters

None

Return

None

Remark

None

7 How to use PMIO Driver

The PMIO driver is initially set by EarlyWakeUp (Pre init) and Init.

7.1 Set Configuration

PMIO driver can be used in configuration flags for power functions. You can see the following flags in "sources/dev.drivers/pmio/pmio.h".

```
#define PMIO_CONF_DEBUG_ALONE
//#define PMIO_CONF_PREVENT_LEAK
//#define PMIO_CONF_DET_BU
#define PMIO_CONF_DET_ACC
#define PMIO_CONF_DET_CAN
#define PMIO_CONF_STR_MODE
//#define PMIO_CONF_MISC_DEV_FUNC
```

To support the suspend scenario provided by Telechips, you must enable PMIO's STR configuration flag as follows:

```
#define PMIO_CONF_STR_MODE
```

To do a sample test for power, you must enable PMIO's MISC DEV configuration flag as follows:

```
#define PMIO_CONF_MISC_DEV_FUNC
```

Refer to Chapter 7.2 for a detailed explanation of *PMIO_CONF_MISC_DEV_FUNC*.

A description of each configuration flag is as follows:

Value	Description
PMIO_CONF_DEBUG_ALONE	[ON] Print errors and debug log all the time. (Not use a LOG_D/E console.)
PMIO_CONF_PREVENT_LEAK	[ON] Prevent power current leakage when system enters power-down or STR mode. This setting is applied to Evaluation Board (EVB) provided by Telechips.
PMIO_CONF_DET_BU	[ON] BU detection possible through GPIOK port
PMIO_CONF_DET_ACC	[ON] ACC detection possible through GPIOK port
PMIO_CONF_DET_CAN	[ON] CAN detection possible through GPIOK port
PMIO_CONF_STR_MODE	[ON] Run in a suspend and power-down scenario, not in normal power-down scenario. ■ Need to support H/W suspend mode. ■ Need to support S/W Mailbox driver. ■ Need to support S/W Timer driver. ■ Need to support S/W AP's suspend mode.
PMIO_CONF_MISC_DEV_FUNC	[ON] Run miscellaneous sample test function ■ Support Backup RAM test

Value	*Description*
	■ *Support Cold Reset test (in AP's DRAM test scenario)* ■ *Support STR iteration test* ■ *Support standalone STR test without AP suspend* *Refer to Chapter 7.2 Set Test Configuration for a detailed explanation of PMIO_CONF_MISC_DEV_FUNC.*

7.2 Set Test Configuration

This chapter is only for test. The operation of this setting is not guaranteed by Telechips.

PMIO driver can be used in test configuration flags for power test functions.
When performing the sample test for power,

1. You should enable PMIO's MISC DEV configuration flag as follows: (sources/dev.drivers/pmio/pmio.h)

```
#define PMIO_CONF_MISC_DEV_FUNC
```

2. You can enable one of the test configurations below: (sources/dev.drivers/pmio/tcc805x/pmio_dev.c)

```
#ifdef PMIO_CONF_MISC_DEV_FUNC
    //#define PMIO_FEATURE_DEV_TEST_BACKUP_RAM
    //#define PMIO_FEATURE_DEV_TEST_STR_ITERATION
    //#define PMIO_FEATURE_DEV_TEST_CRST
    //#define PMIO_FEATURE_DEV_TEST_ALONE_STR_WITH_AP_BOOT_READY
#endif
```

A description of each configuration flag is as follows:

Value	Description
PMIO_FEATURE_DEV_TEST_BACKUP_RAM	[En] This flag performs a sample test about backup RAM read/write. At power-down, a specific value is written to the backup RAM and you can check this value at wake-up. (If wake-up is operated by BU, the backup RAM is cleared.)
PMIO_FEATURE_DEV_TEST_STR_ITERATION	[En] This flag performs a sample test about repeat STR/wakeup. If you enter "pmio test 1" in MCU console as command, MCU requests STR message to APs without ACC signal. And then the system automatically wake-up by the AP's RTC. This sequence allows you to repeatedly test the STR mode. ■ Need to support S/W Mailbox driver. ■ Need to support AP's STR mode with RTC wake-up. ■ Need to support MCU's STR mode.
PMIO_FEATURE_DEV_TEST_CRST	[En] This flag performs a sample test about cold reset. When CA72 and CA53 send a reset request message, PMIO immediately performs a cold reset. ■ Need to support S/W Mailbox driver.
PMIO_FEATURE_DEV_TEST_ALONE_STR_WITH_AP_BOOT_READY	[En] This flag performs a sample test about MCU alone STR entry This test can be operated in conjunction with Sequence Boot Scenario. The Sequence Boot Scenario is a boot scenario that

Value	*Description*
	controls the AP's boot run. *This test uses Sequence Boot Scenario to check the AP's boot status.* *In the STR entry situation,* *If AP's boot is OK, MCU makes a suspend request to AP through MBOX communication.* *If AP's boot is not OK, MCU enters the STR state regardless of the AP.* ■ *Need to support STR Mode.* ■ *Need to support S/W Mailbox driver.* ■ *Need to support Sequence Boot Scenario (MCU's BCTRL app and AP's BL1 BCTRL).*

7.3 Pre-Initialize PMIO Driver

EarlyWakeUp can be used in "*startup.S*" file as follows when you need faster configuration than BSP initialization.

```
    bl  vfp_init
    bl  PMIO_EarlyWakeUp
    b   cmain
    b   .
```

7.4 Initialize PMIO Driver

```
{
    PMIO_Init();
}
```

7.5 Power Down System

```
{
    PMIO_PowerDown(PMIO_MODE_POWER_DOWN);
}
```

7.6 Use BackupRam

```
{
    PMIO_SetBackupRam(PMIO_VA_BACKUP_RAM_ADDR, 1UL);
    PMIO_GetBackupRam(PMIO_VA_BACKUP_RAM_ADDR);
    PMIO_ClearBackupRAM();
}
```

7.7 Get Boot Reason

```
{
    PMIOReason_t reason;
    reason = PMIO_GetBootReason();
    printf("boot reason is %d", reason);
}
```

8 STR

This chapter describes the STR sequence in PMIO driver (PMIO STR sequence) and helps to implement a new power sequence in MCU. If you need to change the STR sequence, it is recommended to create a new power sequence by referring to the PMIO STR sequence described in this chapter. Because the PMIO STR sequence is designed according to the suspend scenario provided by Telechips.

STR HW operation and the PMIO STR sequence are guaranteed by Telechips, but modified STR sequence by the user is not guaranteed. Care must be taken to implement the new sequence.

8.1 Suspend Entry

The STR pin of EVB is GPIO_K12. The STR pin may vary depending on the board that you are using.
The following is a code of entering the STR mode on the PMIO STR sequence.

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
....

#define PMIO_VA_STR_DET_PORT          (PMIO_VA_BIT12)

....

{
    ....
    PMIO_PowerDown(PMIO_MODE_STR);
}
```

8.1.1 Annex: How to Manually Enter Suspend Mode

If you want to enter suspend mode without using the PMIO STR sequence,

1. Set the STR pin to High and keep the status.
2. Power down by **`PMIO_PowerDown(PMIO_MODE_POWER_DOWN);`**

STR Pin control is possible with the **`PMIO_SetPort()`** function.

The following is an example of entering Suspend mode without the PMIO STR sequence.

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
...

...
{
    ....

    PMIO_SetPort(PMIO_PORT_SEL_PMGPIO_OUT_H, PMIO_VA_BIT12);

    PMIO_PowerDown(PMIO_MODE_POWER_DOWN);
}
```

8.2 Communication

PMIO driver uses Mailbox to communicate with CA72/CA53 core. This chapter describes how to communicate with CA72/CA53 core by using MBOX driver in R5 MCU.

For more information on the APIs of the MBOX driver, refer to "*TCC805x MCU BSP-API Specification for Mailbox*".
For more detailed operation of PMIO, refer to PMIO driver code located in mcu_bsp/sources/dev.drivers/pmio/*

8.2.1 Open Mailbox

It is recommend to open Mailbox in the MCU initialization step, as used by **PMIO_STR_OpenMsg()** function in **PMIO_Init().**

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
void PMIO_Init
(
    void
)
{
    PMIO_STR_OpenMsg();
}
```

To implement the new code that you want to use, includes the MBOX driver header as follows.
Open the MBOX driver with the same name as the CA72/CA53 side. (The suspend scenario of Telechips uses "**POWER0**".)
For more information on the APIs of the MBOX driver, refer to "*TCC805x MCU BSP-API Specification for Mailbox*".

The following is an example of opening the MBOX with "**POWER0**".

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
#include <mbox.h>

static MBOXDvice_t *    gPmioMboxA72;
static MBOXDvice_t *    gPmioMboxA53;

static void PMIO_STR_OpenMsg
(
    void
)
{
    int8            cMboxOpenName[7];
    cMboxOpenName[0] = 'P';
    cMboxOpenName[1] = 'O';
    cMboxOpenName[2] = 'W';
    cMboxOpenName[3] = 'E';
    cMboxOpenName[4] = 'R';
    cMboxOpenName[5] = '0';
    cMboxOpenName[6] = '\0';

    gPmioMboxA72  = MBOX_DevOpen(MBOX_CH_CA72MP_NONSECURE, cMboxOpenName, 0U);
    gPmioMboxA53  = MBOX_DevOpen(MBOX_CH_CA53MP_NONSECURE, cMboxOpenName, 0U);
}
```

8.2.1.1 Annex: Create Task

If you successfully open Mailbox, create a task that can process messages.

For creating task API, it is beyond the scope of this document, refer to "*TCC805x MCU BSP-API Specification for System Adaptation Layer*".

PMIO processes messages by creating **PMIO_STR_RecvMsgTask()** and **PMIO_STR_SendMsgTask()** tasks.

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
static void PMIO_STR_OpenMsg
(
    void
)
{
        ....


        if((SAL_TaskCreate(&uiPmioReadMsgTaskId, \
                           (const uint8 *)"PMIO_STR_RecvMsgTask", \
                           (SALTaskFunc)&PMIO_STR_RecvMsgTask, \
                           &(uiPmioReadMsgTaskStack[0]), \
                           (uint32)PMIO_VA_MSG_TASK_STACK_SIZE, \
                           SAL_PRIO_POWER_MANAGER, \
                           NULL)) != SAL_RET_SUCCESS)
        {
            PMIO_E("pmio create task FAIL.\n");
        }

        if((SAL_TaskCreate(&uiPmioSendMsgTaskId, \
                           (const uint8 *)"PMIO_STR_SendMsgTask", \
                           (SALTaskFunc)&PMIO_STR_SendMsgTask, \
                           uiPmioSendMsgTaskStack, \
                           PMIO_VA_MSG_TASK_STACK_SIZE, \
                           SAL_PRIO_POWER_MANAGER, \
                           NULL)) != SAL_RET_SUCCESS)
        {
            PMIO_E("pmio create task FAIL.\n");
        }

}


static void PMIO_STR_RecvMsgTask
(
    void *                          pArg
)
{
    ....
}


static void PMIO_STR_SendMsgTask
(
    void *                          pArg
)
{
    ....
}
```

8.2.1.2 Annex: Create Event

You can also use the Event function for efficient task management.

A detailed description of the Event function is beyond the scope of this document, so refer to "*TCC805x MCU BSP-API Specification for System Adaptation Layer*".

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
static void PMIO_STR_OpenMsg
(
    void
)
{
....

        if ((SAL_EventCreate((uint32 *)&gRecvMsgTaskEventId, \
                        (const uint8 *)"msg signal event created", 0)) != SAL_RET_SUCCESS)
        {
            PMIO_E("pmio create event FAIL.\n");
        }

}
```

8.2.2 Receive Message

You can receive a message like **PMIO_STR_RscvMsgTask().**

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
static void PMIO_STR_RecvMsgTask
(
    void *                          pArg
)
{
    uint32          uiCmd[6];
    PMIOMsg_t       tMsgType;

    (void)MBOX_DevRecvCmd(gPmioMboxA53, uiCmd);

    if(uiCmd[0]==0x1)
    {
        ....
    }

}
```

The buffer for receiving MBOX messages requires 24 bytes.

The message format supported by Telechips is composed of 4 CMDs of 8 bits each as shown in Figure 8.1.

For more information on the APIs of the MBOX driver, refer to "*TCC805x MCU BSP-API Specification for Mailbox*".

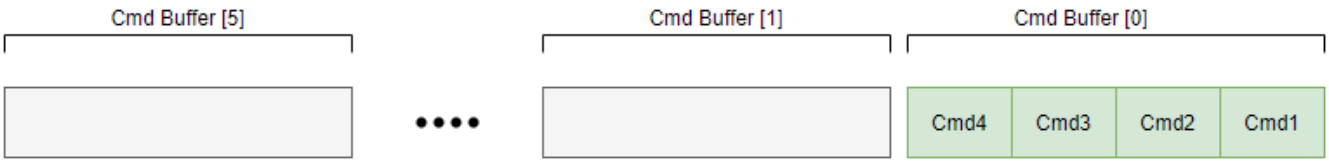

Figure 8.1 MBOX Message Format

8.2.2.1 Annex: Parsing Message Data

PMIO analyzes the received message data using **PMIO_STR_GetMsgData()** function.

For customization, you can refer to this chapter and perform a new parsing process.
Or if you decide that parsing is unnecessary, you can use the message directly without parsing function. In this case, skip this chapter.

The **PMIO_STR_GetMsgData()** function classifies the received message data as follows and determines message type. If you need to extend the message length, customize and use this function.

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
static void PMIO_STR_RecvMsgTask
(
    void *                          pArg
)
{
    uint32          uiCmd[6];

    ....

    tMsgType = PMIO_STR_GetMsgData(uiCmd);
}


static PMIOMsg_t PMIO_STR_GetMsgData
(
    uint32                          *puiMsg
)
{
    uint8 uiCmd1;
    uint8 uiCmd2;
    uint8 uiCmd3;

    uiCmd1 = (uint8) ((puiMsg[0] >> 0UL ) & 0xffUL);
    uiCmd2 = (uint8) ((puiMsg[0] >> 8UL ) & 0xffUL);
    uiCmd3 = (uint8) ((puiMsg[0] >> 16UL) & 0xffUL);

}
```

8.2.3 Send Message

You can send a message like **PMIO_STR_SendMsgTask().**

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
static void PMIO_STR_SendMsgTask
(
    void *                              pArg
)
{
     MBOX_DevSendCmd(gPmioMboxA72, uiSendBuf);
}
```

8.2.3.1 Annex: Create Message Data

PMIO creates the message to be sent using the **PMIO_STR_SetMsgData()** function.

For customization, you can refer to this chapter and perform a new message data creation process.
Or if you decide that creating message data is unnecessary, you can configure the MBOX buffer directly without a message creator. In this case, skip this chapter.

The **PMIO_STR_SetMsgData()** function create the message to be sent as follows. If you need to extend the message length, customize and use this function.

```
PMIOMsg_t       gSendMsgType = PMIO_MSG_NOT_SET

static void PMIO_STR_SendMsgTask
(
    void *                              pArg
)
{
    uint32          uiCmd[6];

    ....

    PMIO_STR_SetMsgData(gSendMsgType, uiCmd);
}

static void PMIO_STR_SetMsgData
(
    PMIOMsg_t                           uiMsgType,
    uint32                              *puiMsg
)
{
    uint32 uiCmd;

    uiCmd = (   (gMsgDataList[(uint32)uiMsgType].msgCmd1 << 0  ) |
                (gMsgDataList[(uint32)uiMsgType].msgCmd2 << 8  ) |
                (gMsgDataList[(uint32)uiMsgType].msgCmd3 << 16 )    );

        puiMsg[0] = uiCmd;
}
```

8.3 Sequence

This chapter describes PMIO STR sequence as shown in Figure 8.2 to Figure 8.9.

For more detailed operation, refer to PMIO driver code located in mcu_bsp/sources/dev.drivers/pmio/*

8.3.1 Power-on/ACC ON Sequence

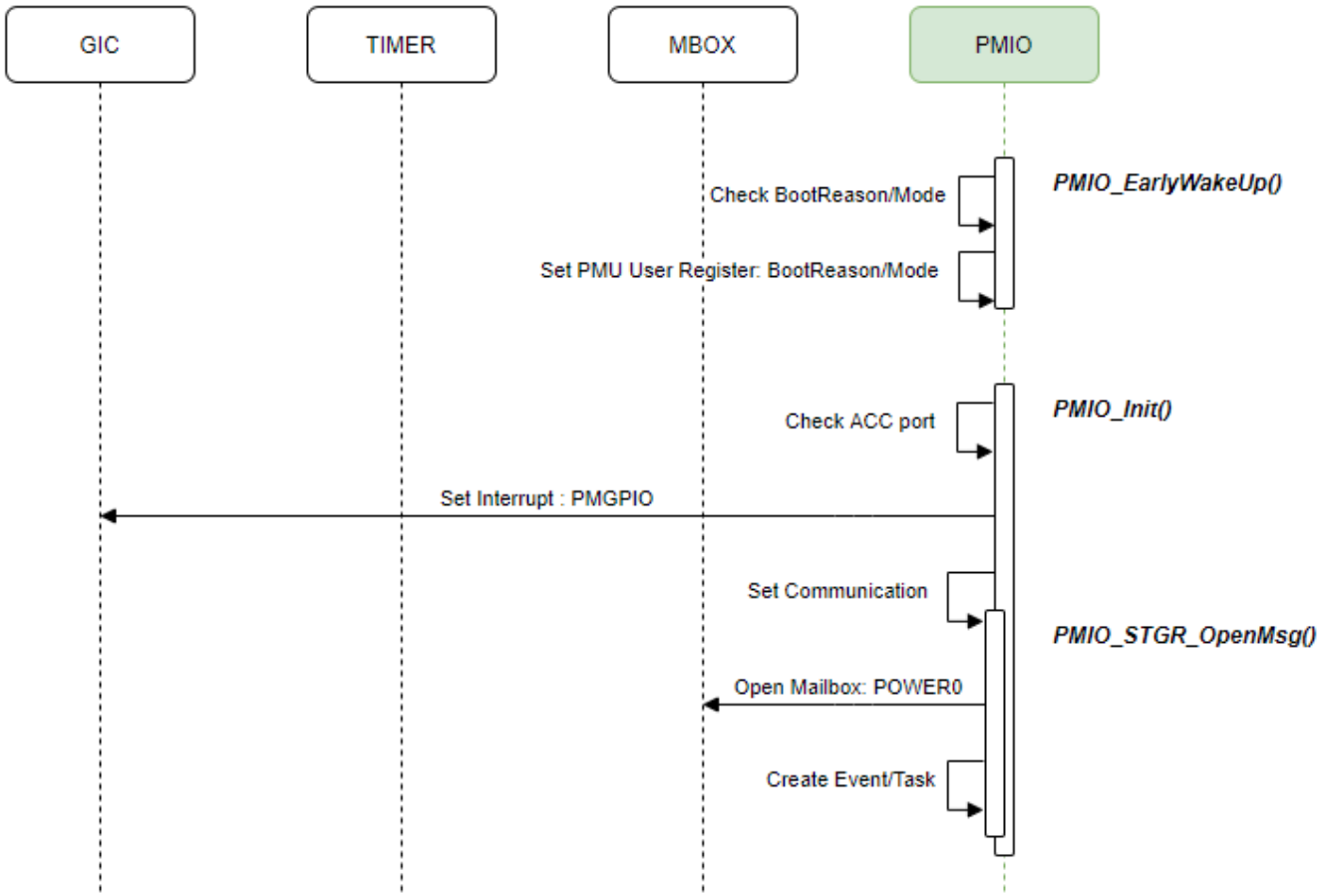

Figure 8.2 Power-on/ACC ON

8.3.1.1 TIMEOUT Sequence (Power-on without ACC ON)

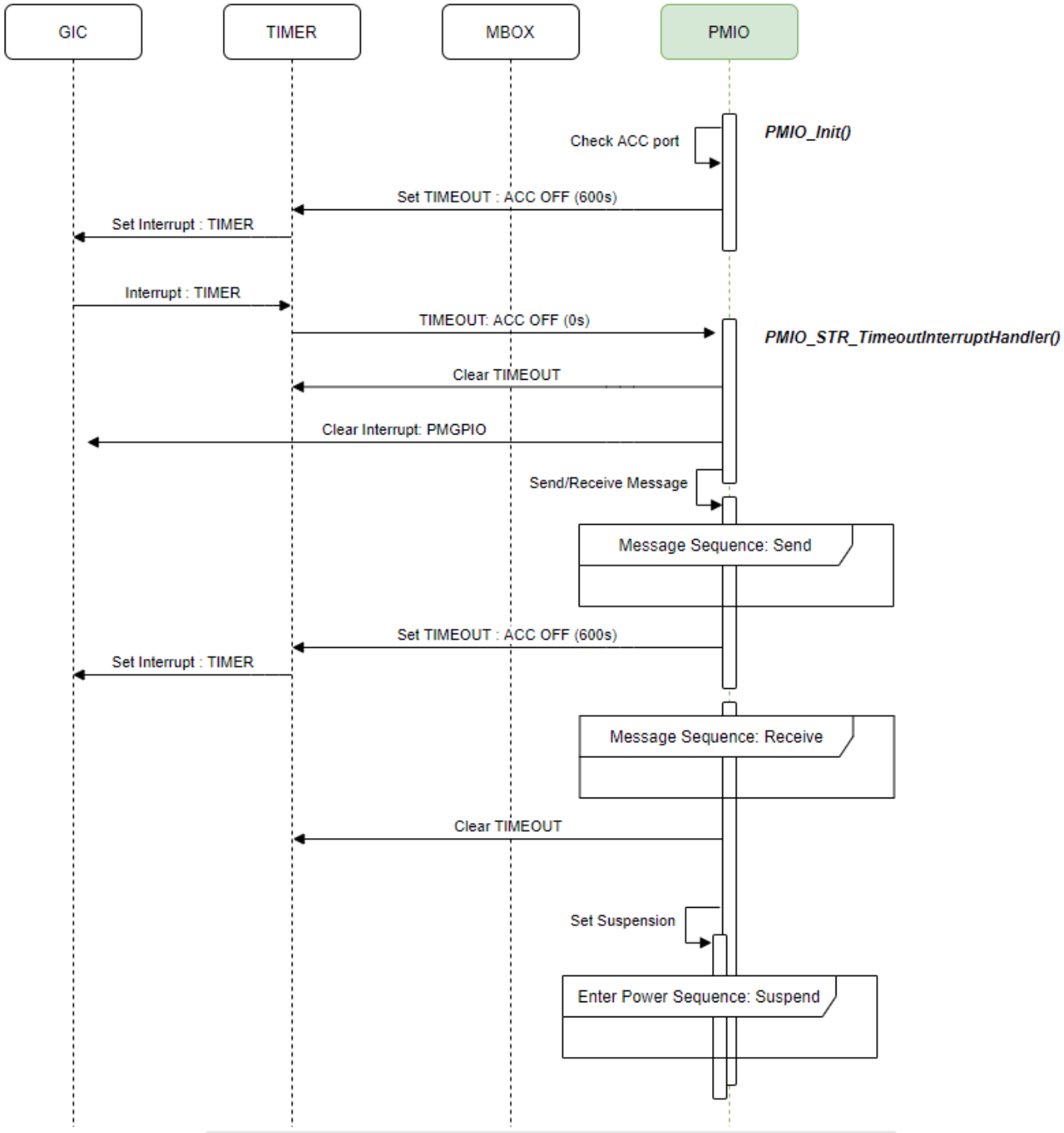

Figure 8.3 Timeout (Power-on without ACC ON)

8.3.2 ACC OFF Detection Sequence

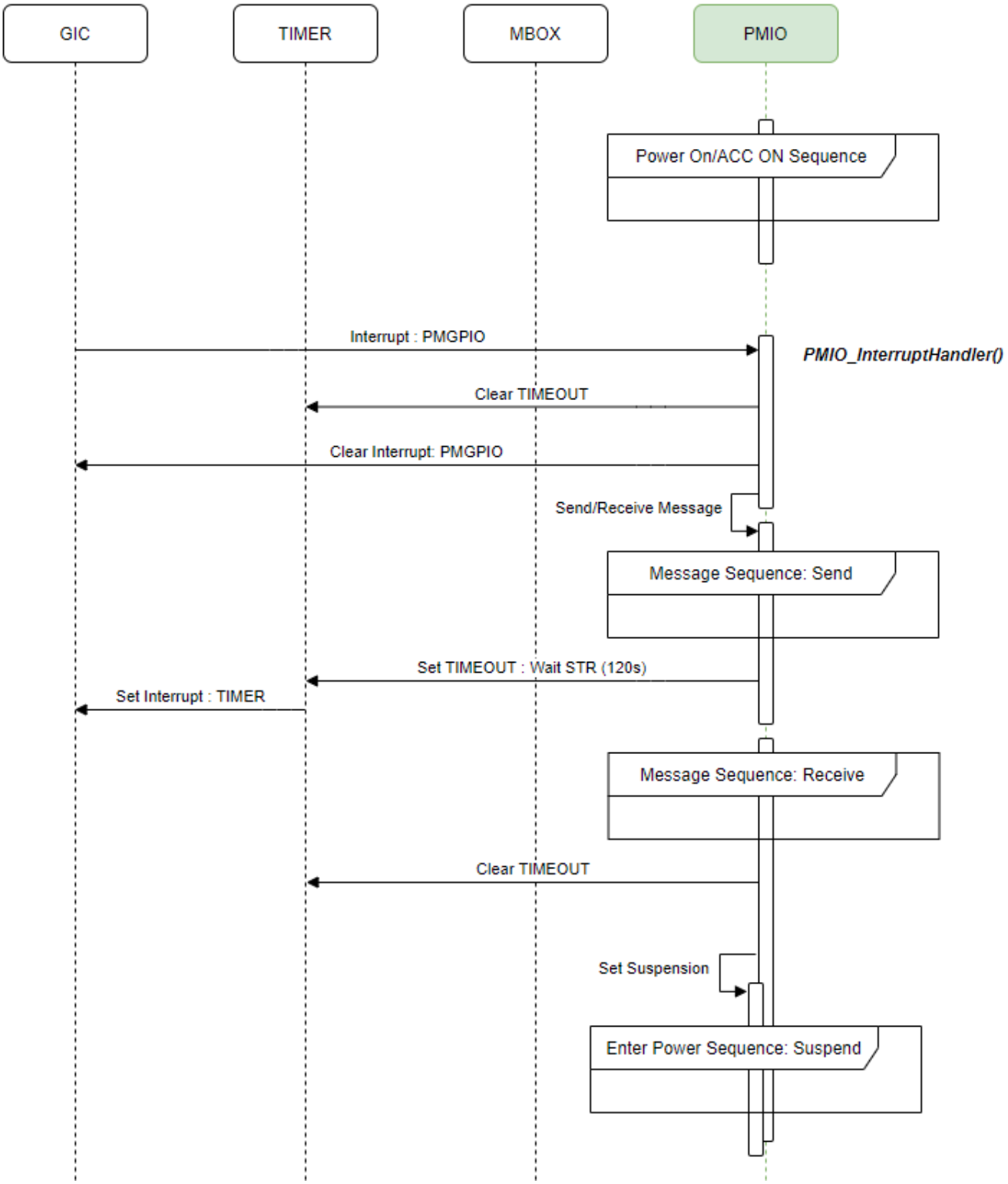

Figure 8.4 ACC OFF Detection

8.3.2.1 TIMEOUT Sequence (No STR Message Received)

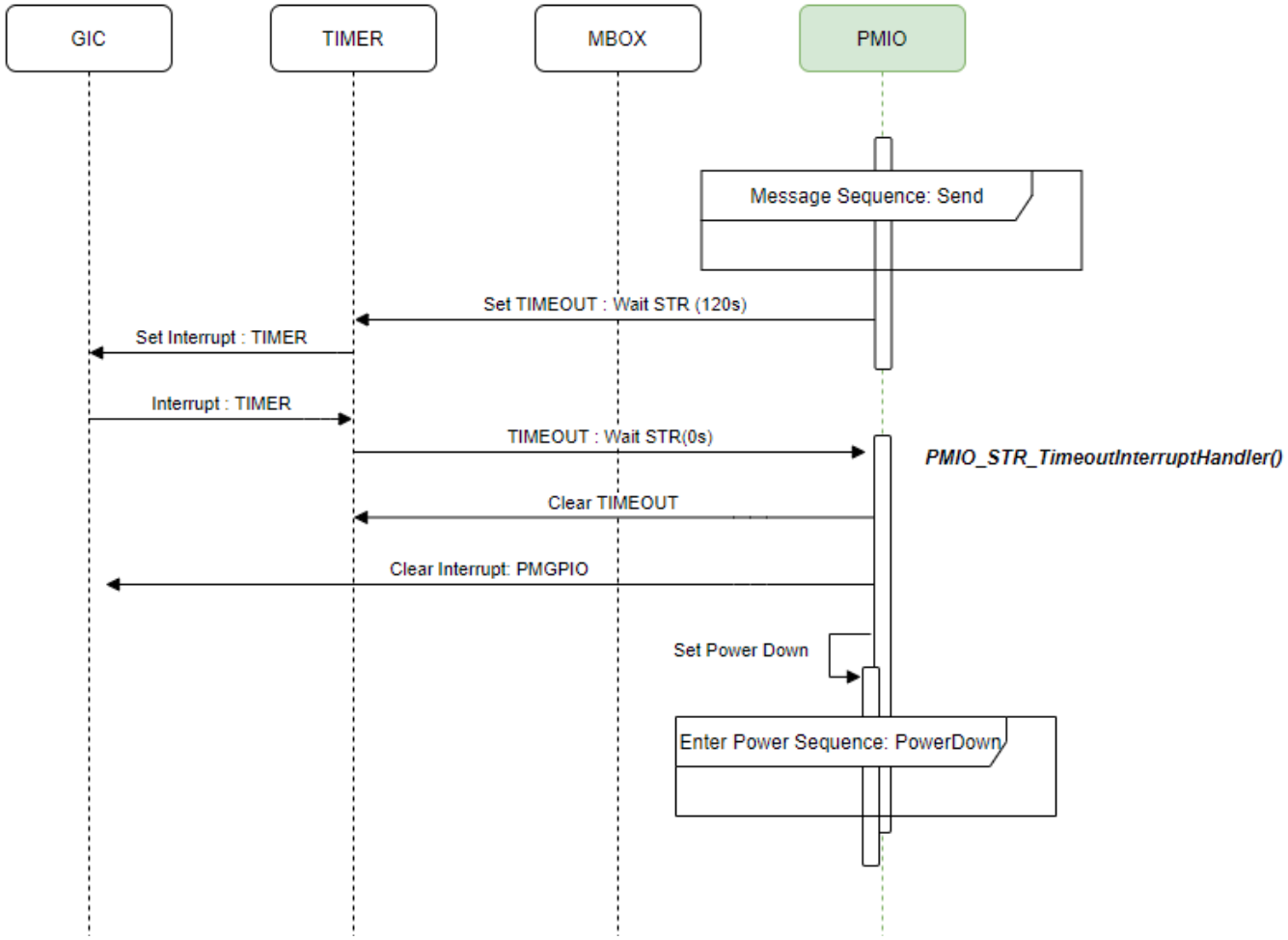

Figure 8.5 Timeout (No STR Message Received)

8.3.3 Enter Power Sequence

8.3.3.1 Power-down

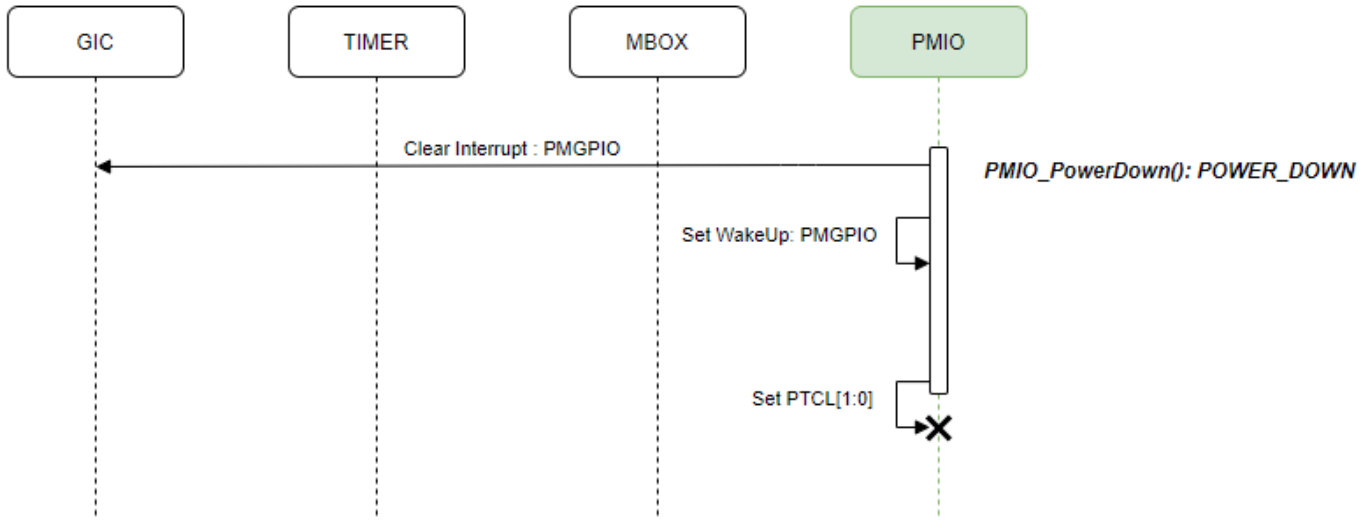

Figure 8.6 Power-down

8.3.3.2 STR

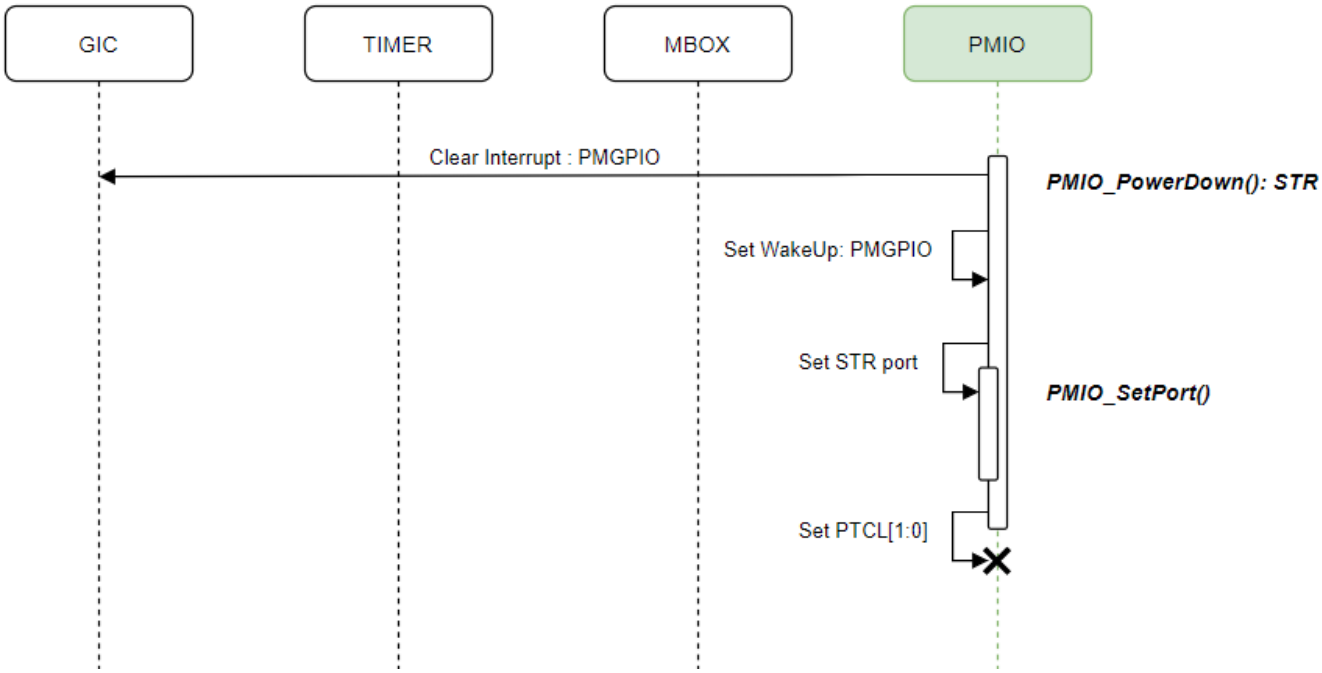

Figure 8.7 STR

8.3.4 Message Sequence

8.3.4.1 Send

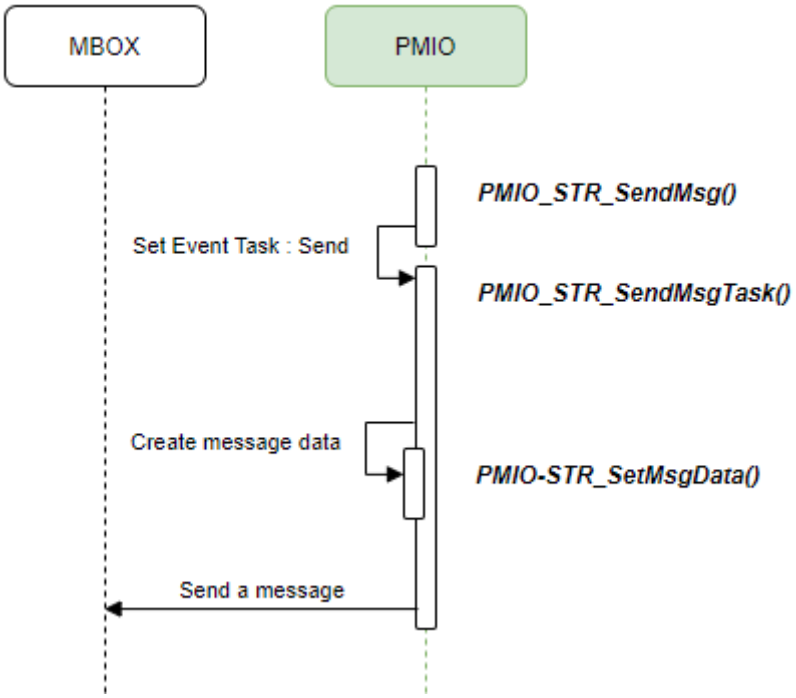

Figure 8.8 Send

8.3.4.2 Receive

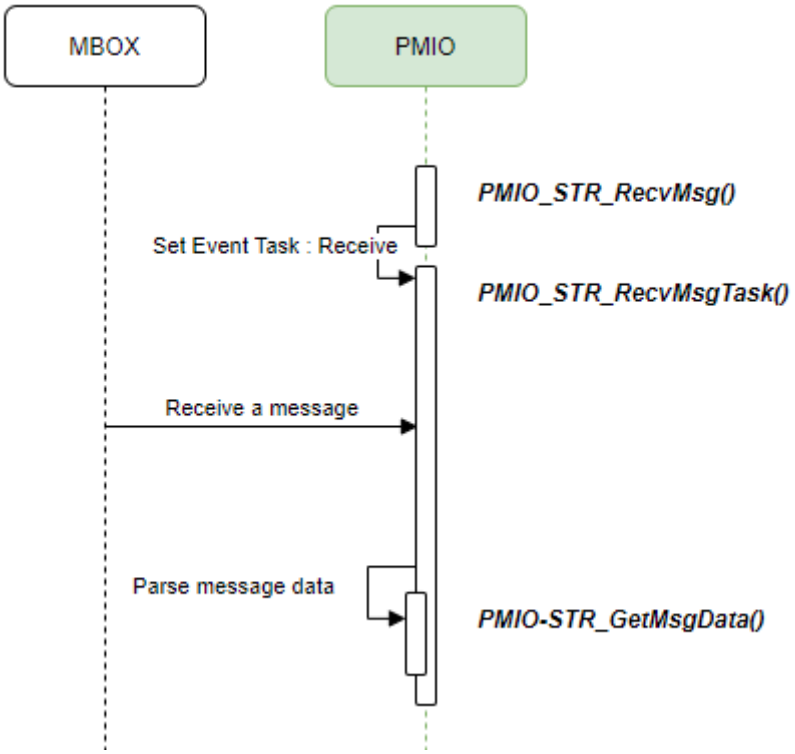

Figure 8.9 Receive

8.4 Simple re-implementation PMIO STR Sequence

To implement a new power sequence, it is recommended to disable the PMIO STR sequence.

However, if you want to re-implement the sequence based on the PMIO STR sequence, this chapter will help.

Important: Modified operation by the user is not guaranteed by Telechips, even based on the PMIO STR sequence.

8.4.1 Enable STR Mode

- mcu_bsp/sources/dev.drivers/pmio/pmio.h

```
/*
*******************************************************************************************************
*                                          DEFINITIONS
*******************************************************************************************************
*/

...

#define PMIO_CONF_STR_MODE

...

/* ================================================== CONFIGURATION for PMIO FUNCTION*/
```

8.4.2 Modify Interrupt Processing

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
static void PMIO_InterruptHandler
(
    void *                              pArg
)
{
    [RE_IMPLEMENTATION]
}
```

8.4.3 Modify Processing of Receive Message

- mcu_bsp/sources/dev.drivers/pmio/tcc[xxxx]/pmio_dev.c

```
static void PMIO_STR_RecvMsgTask
(
    void *                          pArg
)
{
    while(1)
    {

        ....

        if(uiMboxAPReady != 0UL)
            {
                if(gMsgMainCore == 53UL)
                {
                    (void)MBOX_DevRecvCmd(gPmioMboxA53, uiCmd);
                }

                if(gMsgMainCore == 72UL)
                {
                    (void)MBOX_DevRecvCmd(gPmioMboxA72, uiCmd);
                }

                [RE_IMPLEMENTATION]

                ....
            }

            ....
    }
}
```

9 REFERENCES

[1] TCC805x Full Specification

[2] Contact Telechips for more details: sales@telechips.com

10 REVISION HISTORY

Rev. 1.30: 2021-06-01

- Updated
  - Chapter 2: Table 2.1

- Added
  - Chapter 8: STR

Rev. 1.20: 2021-03-24

- Updated
  - Chapter 3.2: Description
  - Chapter 5.1: Remark
  - Chapter 6.7: Return
  - Chapter 6.8: Description
  - Chapter 7.1: Declaration and description

- Added
  - Chapter 4.4: Declaration and description
  - Chapter 4.5: Declaration and description
  - Chapter 5.2: Declaration and description
  - Chapter 6.9-6.12: Declaration and description
  - Chapter 7.2: Declaration and description.

Rev. 1.11: 2021-01-22

- Updated
  - Chapter Simple Description of Source Files3.2: Description
  - Chapter 5.1: Declaration and description
  - Chapter 5.3: Declaration and description
  - Chapter 5.5: Declaration
  - Chapter 7.1: Path and value description

Rev. 1.10: 2020-11-27

- Added
  - Chapter 7.1 Set Configuration

- Deleted
  - Chapter 5.1 PMIO Configuration

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.
No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.
This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.
Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:
"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit http://mp3licensing.com".

For customers who use other firmware of mp3:
"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit http://mp3licensing.com".

For customers who use Digital Wave DRA solution:
"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:
"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:
"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:
"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:
"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."